Get In Line

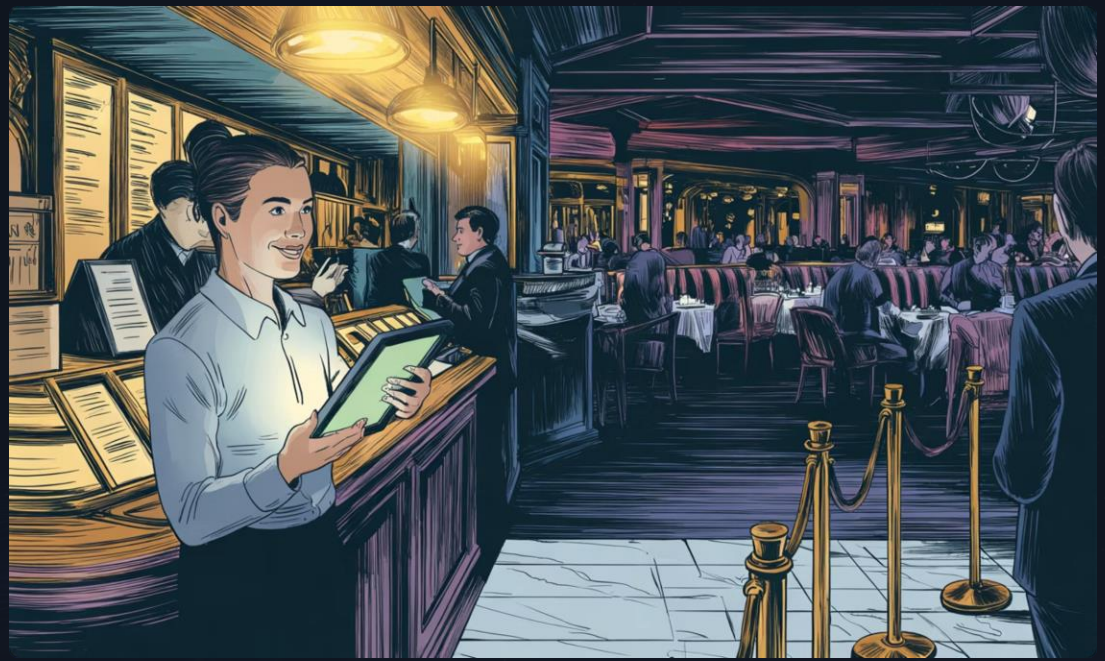
GIL is a real-time event waitlist management platform designed to streamline entry management and room capacity for high-volume, multi-venue environments.

TEAM 12 · CS 4485 MAY 5, 2026

Kyle Albuquerque · Angela Lopez · Disha Shetty · Neel Vavilapalli

Supervisor: Professor Sridhar Alagar · Project Coordinator: Adarsh Gella

Introduction



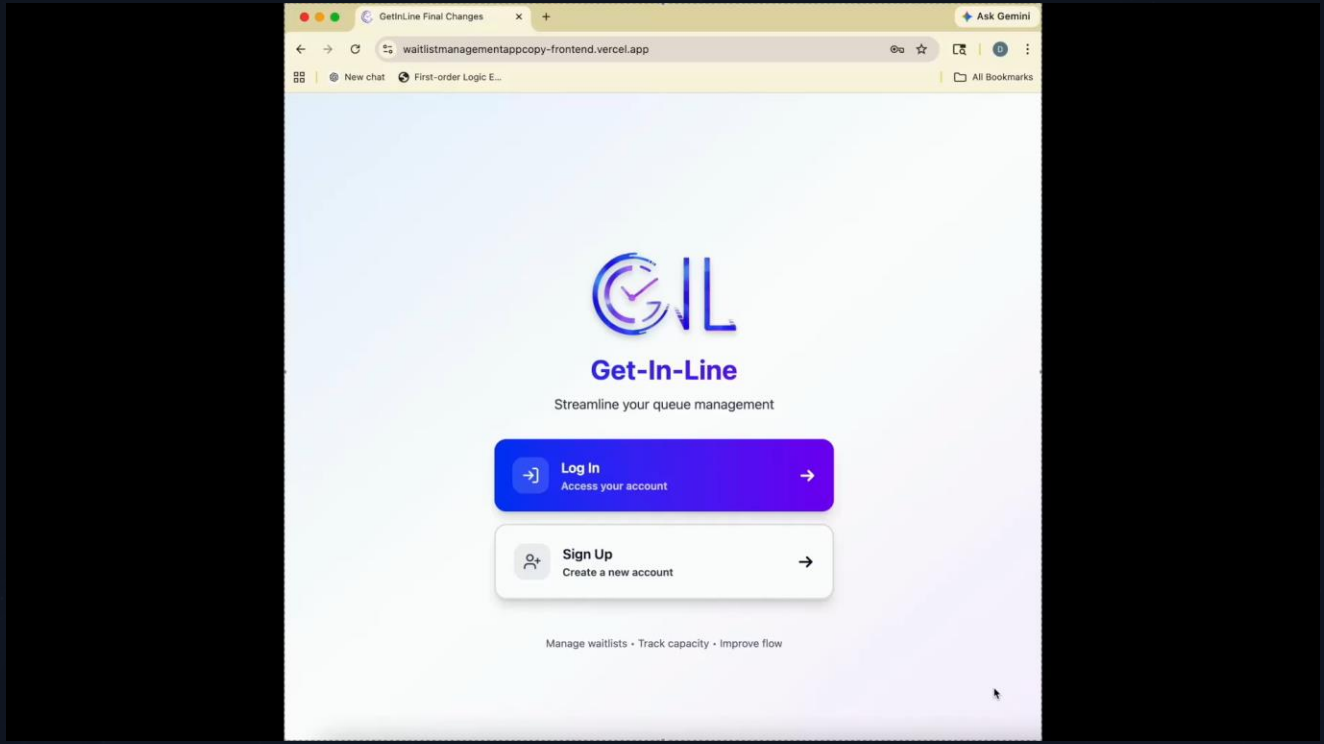
The Problem

Current waitlist systems rely on manual processes and static estimates — leading to inaccurate wait times, poor customer experience, and no support for diverse event types.

The Solution

Get In Line is a full-stack web app enabling businesses to create and manage events while users join waitlists with dynamically calculated wait time estimates. Supports attendance-based, capacity-based, and table-based events.

Demo


GetInLine Final Changes
Ask Gemini
waitlistmanagementappcopy-frontend.vercel.app
New chat
First-order Logic E...
All Bookmarks
GIL
Get-In-Line
Streamline your queue management
Log In
Access your account
Sign Up
Create a new account
Manage waitlists • Track capacity • Improve flow

Tech Stack & Architecture

Frontend

Vite + React + TypeScript — local-first with localStorage persistence

Backend

Node.js/Express orchestrator + FastAPI waitlist heuristics service

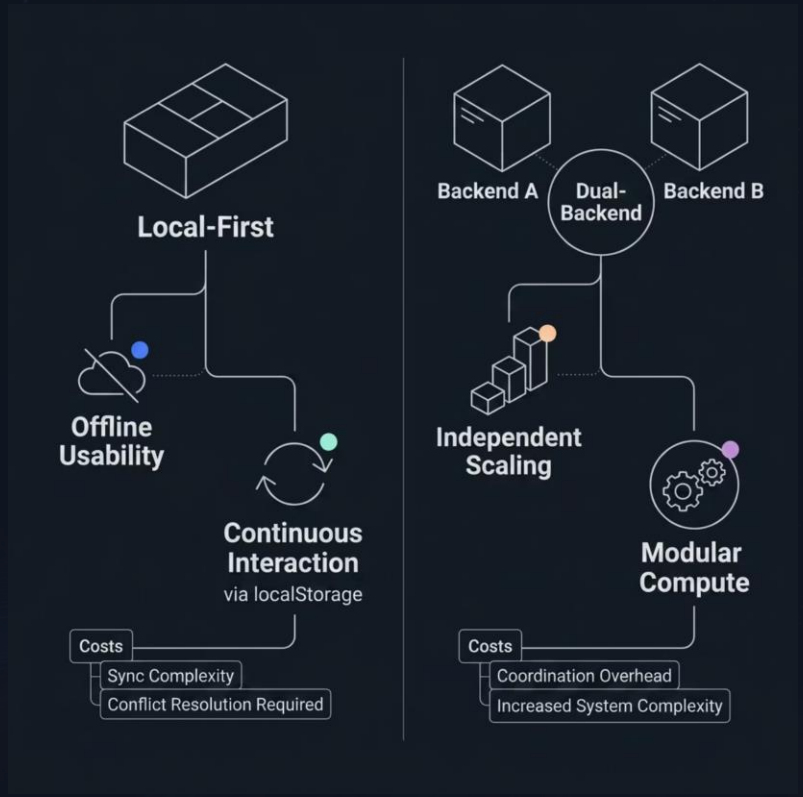
Database

Supabase — centralized auth, real-time data, source of truth

Deployment

Vercel (frontend) + Render (backend services)

Design Choices & Trade-offs



Key Decisions

- **Local-first frontend:** Optimistic updates via localStorage ensure offline usability but add sync and conflict resolution complexity
- **SERVER_WINS strategy:** Prioritizes data consistency during reconciliation between local and remote states
- **Two backend services:** Node.js orchestrator + FastAPI heuristics enables independent scaling at the cost of coordination overhead
- Post hog for web analytics

Performance & Takeaways

~1.2ms

Avg Latency

Consistent under load

<3ms

p95 Latency

95th percentile response time

375

Req/sec

Peak throughput from testing VUs

Frontend	Backend
Global Scaling	Single Instance

Future work: multi-instance backend with load balancing, Redis caching, and containerization for large-scale deployments.

Wait Time Prediction Model

$$\text{Estimated Wait} = \left(\sum w_i\right) \times (1 - r_{no-show}) \times t_{avg}$$

What each variable means

- w_i = weight of user i
  (value between 0 and 1 based on recent activity; higher = more active user)
- $\sum w_i$ = total "effective queue size"
  (instead of counting people, we sum their weights)
- $r_{no-show}$ = no-show rate
  (percentage of users expected not to show; updated in real time as people are seated or drop out)
- t_{avg} = average wait time per party
  (dynamic value that updates based on how fast parties are being seated and how long the queue has been open)

Thank you!